

Operations Research

Integer Programming (IP)

Prof. Dr. Martin Bichler

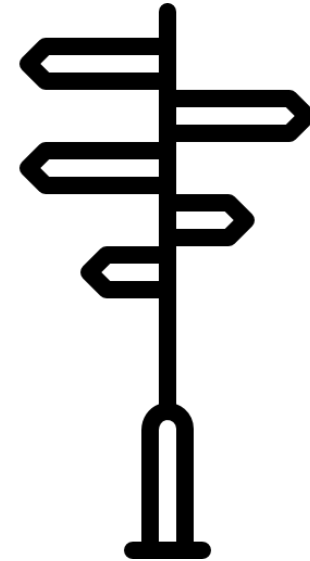
Department of Computer Science

School of Computation, Information, and Technology

Technical University of Munich

Course outline

- **Linear Optimization**
 - Modeling by linear programming and graphical solution
 - Solving linear programs, convexity
 - Simplex algorithm, 2-phase simplex
 - Simplex algorithm in matrix notation
 - Sensitivity analysis, reoptimization
 - Duality theory, zero-sum games
- **Integer Linear Optimization**
 - **Modeling integer programming problems (IPs)**
 - Solving IPs: branch & bound, cutting planes
 - Advanced solution methods: column generation, approximation algorithms
 - Fast solvable integer problems: total unimodularity, matroids
 - Network flow problems
 - Traveling salesperson problem
- **Nonlinear Optimization**
 - Optimization of multidimensional functions
 - Convex optimization



Today you will learn...

- what **modeling possibilities** exist with integer linear programming (IP).
- the difference in **structure** and **runtime complexity** between LP and IP.
- how to formulate well-known and commonplace **optimization models as IP**.



MIP, IP, BIP

- LP: $\max\{c^T x : Ax \leq b, x \geq 0\}$

- MIP: $\max c^T x + h^T y$
 $Ax + Gy \leq b$
 $x \geq 0, y \in \mathbb{N}_0$

- BIP: $\max c^T x$
 $Ax \leq b$
 $x \in \{0,1\}$

Integer programming

Why do you need Integer Programs (IPs)?

- Some decisions are naturally restricted to integer values.
- Binary variables, which are restricted to 0 and 1, model yes/no decisions in mathematical models.
- While it is often easy to round LP solutions for large (production) quantities with low marginal costs, this is not so easy for a few costly decisions.
- “Rounding” binary decisions can lead to poor or infeasible solutions.

Example

TUM is opening a new campus.

It is possible to build 5 dormitories with 100, 500, 400, 300 and 50 dormitory places, respectively.

The construction costs are € 100k, € 400k, € 360k, € 275k and € 75k, respectively.

It is expected that 600 students will study at the campus next year*.

For each student who does not receive a dormitory place, the city will pay € 950.

What should be done to minimize the costs in this problem?

**future years are ignored in this example*

Modeling

- Objective: minimize the costs
 - The city must pay for each building.
 - The city must pay for each student who does not receive a space.
- Constraints:
 - Dormitories must first be built in order to house students.
 - Student housing capacity is limited.
 - The number of students who do not get a place is 600 minus the number of students who get a place.

Summary

Planned dormitories	Capacity	Costs
i	n_i	c_i
1	100	€ 100k
2	500	€ 400k
3	400	€ 360k
4	300	€ 275k
5	50	€ 75k

Demand: 600 students

Costs if a student does not receive a place: € 950

LP vs IP: optimal solutions

Solved in 1 iterations and 0.01 seconds

Optimal objective 4.900000000e+05

x1: 0.0

x2: 1.0

x3: 0.25

x4: 0.0

x5: 0.0

s: 0.0



Optimal solution found (tolerance 1.00e-04)

Best objective 4.950000000000e+05, best bound 4.950000000000e+05, gap 0.0000%

x1: 0.0

x2: 1.0

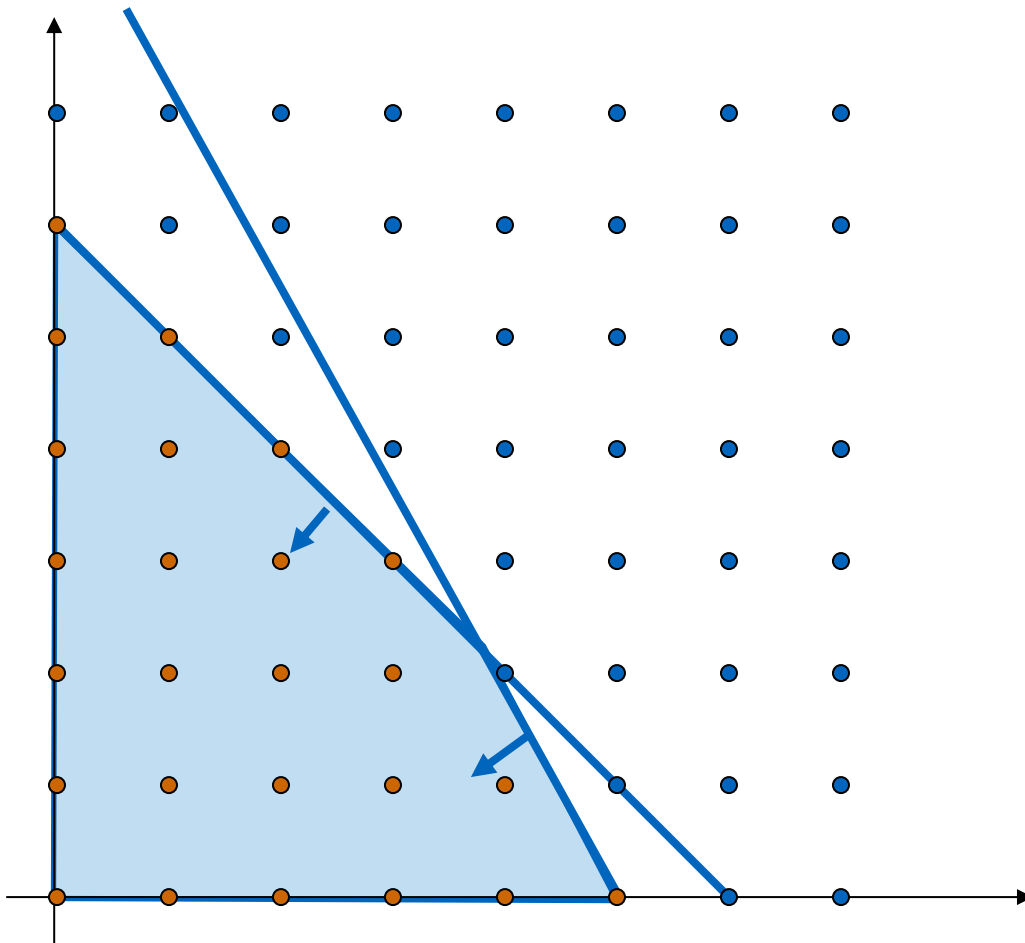
x3: 0.0

x4: 0.0

x5: 0.0

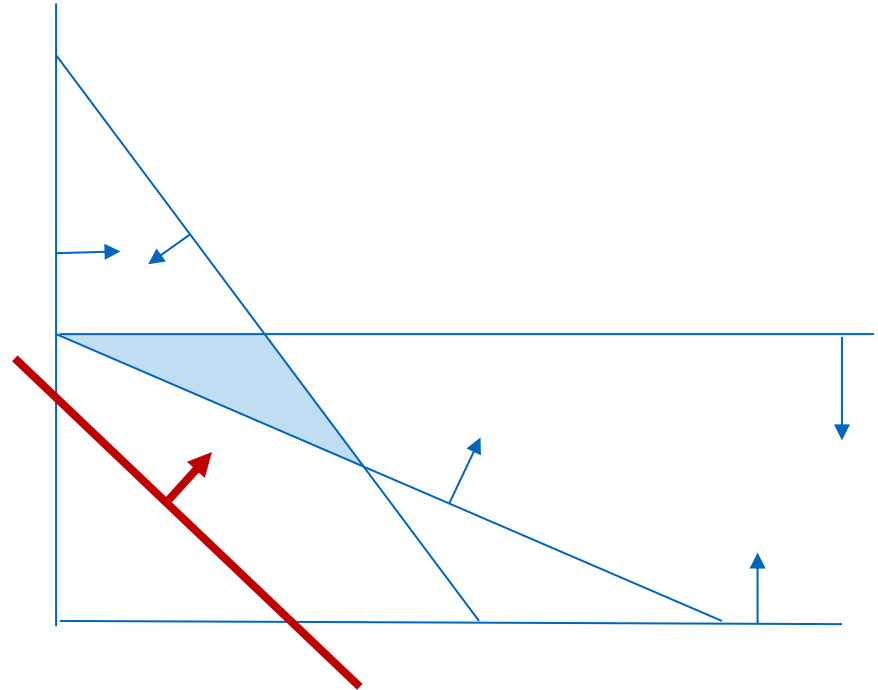
s: 100.0

LP vs. IP: feasible region



LP vs. IP: feasible region

$$\begin{array}{llllll} \max & x_1 & + & x_2 & & \\ \text{s.t.} & & & 2x_2 & \leq & 7 \\ & 7x_1 & + & 16x_2 & \geq & 56 \\ & 4x_1 & + & 3x_2 & \leq & 20 \\ & & & x_1, x_2 & \geq & 0 \end{array}$$



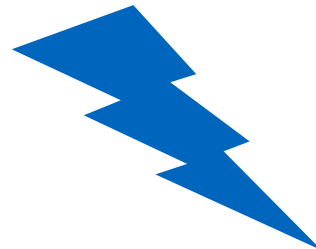
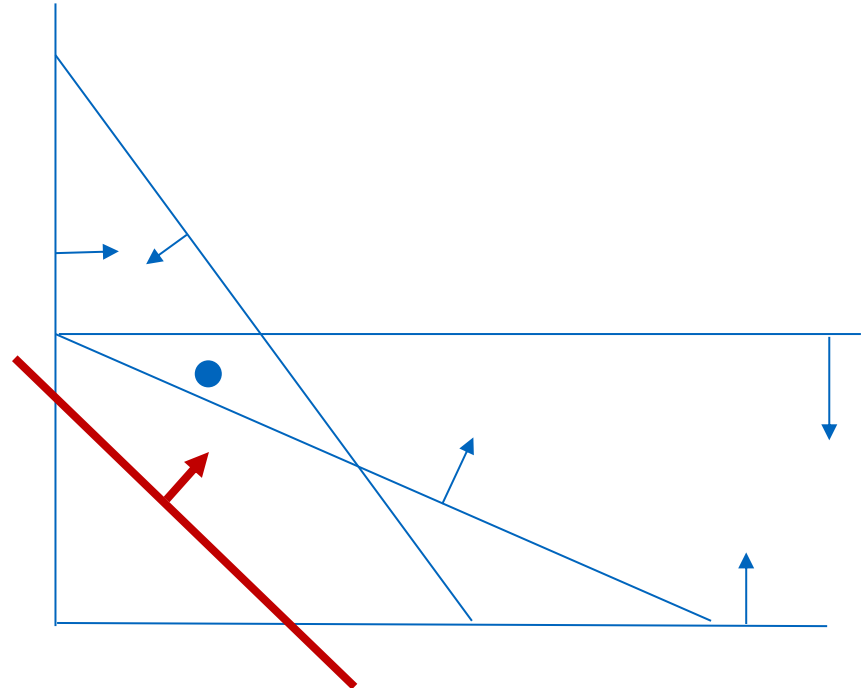
LP vs. IP: feasible region

$$\begin{array}{llllll}
 \max & x_1 & + & x_2 & & \\
 \text{s.t.} & & & 2x_2 & \leq & 7 \\
 & 7x_1 & + & 16x_2 & \geq & 56 \\
 & 4x_1 & + & 3x_2 & \leq & 20 \\
 & & & x_1, x_2 & \in & \mathbb{N}_0
 \end{array}$$

Only one feasible solution: (2,3)

Vertex solutions:

- (0;3.5)
- (2.375; 3.5)
- (3.54;1.95)



Solving the problem using Simplex (or by rounding the Simplex solution using „reasonable“ rounding approaches) leads to bad results.

Modeling with binary variables

- Selection constraints from k values
 y can take only one value from $\{a_1, \dots, a_k\}$
 $y = \sum_j a_j x_j, \sum_j x_j = 1$ and $x_j \in \{0, 1\}$
- Either choose state 1 and 2 or neither: $x_1 \leftrightarrow x_2$
 $x_1 = x_2$ selection constraint with $x_j \in \{0, 1\}$
- If state 1 is selected, then state 2 must also be selected: $x_1 \Rightarrow x_2$

$$x_1 \leq x_2 \Rightarrow$$

0	0		1
0	1		1
1	0		0
1	1		1

Boolean logic vs. mathematical equations

Logical operators	Pure logical statements	Mathematical representation
Logical “OR”	$x_1 \vee x_2 \vee \dots \vee x_n$	$x_1 + x_2 + \dots + x_n \geq 1$
Logical “AND”	$x_1 \wedge x_2 \wedge \dots \wedge x_n$	$x_1 \geq 1;$ $x_2 \geq 1; \dots;$ $x_n \geq 1$
Implication ($\Rightarrow$)	$x_1 \Rightarrow x_2$ $\neg x_1 \vee x_2$	$x_1 \leq x_2$ or $x_2 - x_1 \geq 0$
Equivalence ($\Leftrightarrow$)	$x_1 \Leftrightarrow x_2$ $(\neg x_1 \vee x_2) \wedge (\neg x_2 \vee x_1)$	$x_1 - x_2 \leq 0;$ $x_2 - x_1 \leq 0$

Example: student dorms

How can the following conditions be modeled, assuming $x_i \in \{0,1\}$?

- Dorms 2 and 3 are planned for the same location
– only one of the two can be built. $x_2 + x_3 \leq 1$
- The total cost of the buildings must not exceed € 450k. $\sum c_i x_i \leq 450$
- A maximum of 3 dormitories should be built. $\sum x_i \leq 3$
- Dormitory 2 may only be built if dormitory 4 is built. $x_2 \leq x_4$
- Dormitory 4 may only be built if all students are housed at the end. $600x_4 \leq (600 - s)$

Additional modeling tricks

Modeling threshold values:

- Example: for more than 100 units purchased, there is a discount of 500 €.

$x \in \mathbb{N}_0$: number of produced units

$b \in \{0,1\}$: discount level reached

$$\begin{aligned} 99 + Mb &\geq x \\ 100b &\leq x \end{aligned}$$

Big-M Constraint
(„very large“ number)

- When a product is produced, at least 100 units must be produced (or zero).

$$x \geq 100b$$

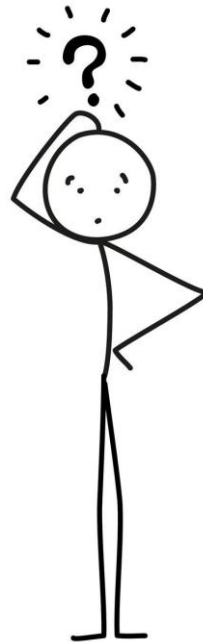
$$x \leq Mb$$

- Even multiplications of variables ($x_1 \cdot x_2$) can be linearized using auxiliary variables (see exercises).

- ... Modeling exercises at: <https://vmbichler48.cs.tum.edu/>

How can you linearize the following non-linear constraint?

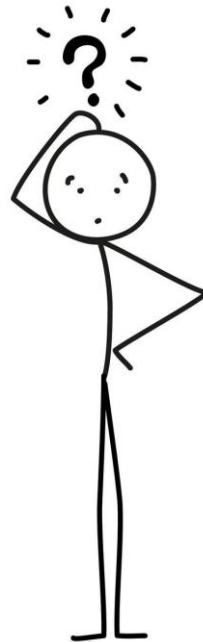
$$\min\{x_1, x_2\} \leq 1$$



How can you linearize the following non-linear constraint?

$$\min\{x_1, x_2\} \leq 1$$

$$x_1 \leq 1 + My_1; x_2 \leq 1 + M(1 - y_1); y_1 \in \{0,1\}$$



Detour: complexity theory

- Objective: estimating the memory and runtime behavior of algorithms
 - evolution of the runtime depending on the size of the program
 - worst case-, average case-, best case analyses
- Auxiliary construct for the analysis of runtime complexity
 - definition of an idealized computer model for the analysis of complexity
 - RAM (random access machine)
 - instruction set similar to assembly (load, store, arithmetic linking of register contents, jumps)
 - infinite set of memory cells storing natural or real numbers
- Analysis of:
 - memory → number of required RAM cells
 - runtime → number of executed RAM instructions

Complexity classes

Three main **complexity classes** are distinguished. The runtime depends on the length of the input n (e.g., number of objects, products, etc.).

An algorithm A is called:

linear $f_A \in O(n)$

e.g. linear search in an unordered list

polynomial $\exists k \in \mathbb{N}$, such that $f_A \in O(n^k)$

e.g. bubble sort

exponential $\exists k \in \mathbb{N}$, such that $f_A \in O(k^n)$

e.g. complete enumeration of the power set

Complexity classes P and NP

P: The set of all problems that a deterministic automaton solves in polynomial time $O(n^k)$.

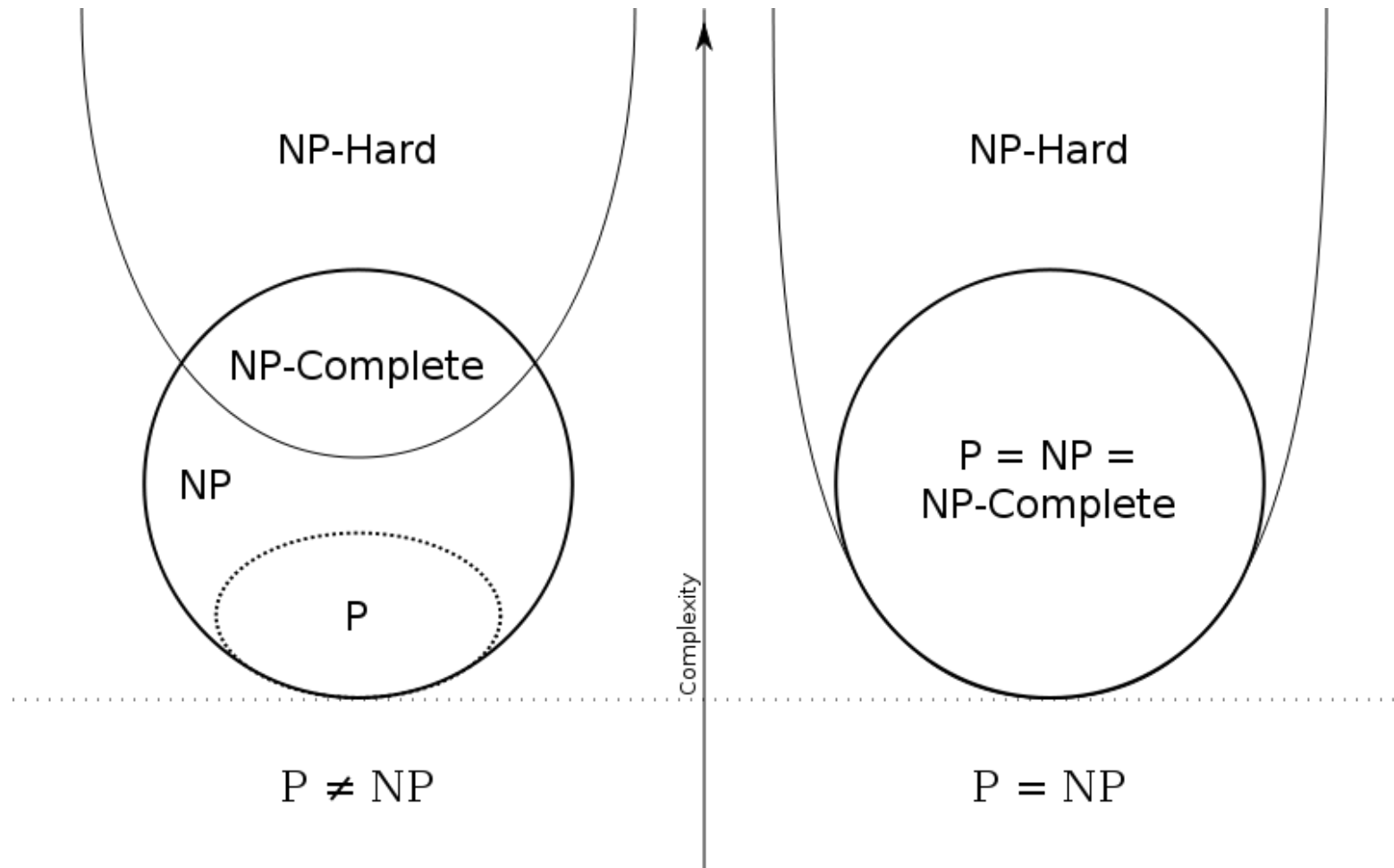
- These problems are efficiently solvable (i.e., in polynomial time).

NP: The set of all problems that a nondeterministic automaton solves in polynomial time.

- Other definition: set of problems whose optimality/feasibility can be checked in polynomial time.

One of the biggest questions in computer science is whether for all problems in NP there also exists a polynomial algorithm that solves the problems efficiently ($P = NP$), or whether there are problems in NP that cannot be decided (solved) in polynomial time.

P vs NP



NP-completeness

A problem A is called **polynomially reducible** to a problem B if there exists a polynomial transformation of an instance I_1 of A into an instance I_2 of B, such that a solution to I_1 is obtained from a solution to I_2 by back transformation.
(B is then at least as difficult as A)

A problem A is called **NP-hard** if every problem from NP can be reduced to A.

A problem A is called **NP-complete** if it is in NP and if every problem from NP can be reduced to it.

→ If a polynomial algorithm were found for just one NP-complete problem, it would be shown that $P = NP$.

Karp's NP-complete problems

- SAT
 - Is there any assignment of values for a logical expression (e.g., circuit) that makes the expression “true”?
- Traveling Salesperson Problem
- Scheduling
- Coloring Problem
 - Can a map be colored using k colors, such that all countries are distinguishable?
- Knapsack problem
- Bin packing problem
- Set cover
- **Integer programming**
- ...

Boolean satisfiability problem (Satisfiability, SAT)

- A boolean function $\phi = f(x_1, x_2, \dots, x_n)$ is satisfiable if there exists an assignment of values for $x_1, x_2, \dots, x_n$, such that $f(x_1, x_2, \dots, x_n) = 1$.
 - $(x \vee y) \wedge (z \vee \neg x \vee \neg y) \wedge (x \vee \neg z)$ is satisfiable, since
 - the assignment $x = 1, y = 0, z = 0$ leads to
 - $(1 \vee 0) \wedge (0 \vee 0 \vee 1) \wedge (1 \vee 1) = 1 \wedge 1 \wedge 1 = 1$.
- **Definition (Boolean satisfiability problem)**
 - **SAT** = $\{ \phi \mid \phi \text{ is a satisfiable function} \}$, i.e.
 - given: boolean function ϕ
 - question: Are there any $x_1, x_2, \dots, x_n$, such that $\phi(x_1, x_2, \dots, x_n) = 1$?
- **Special case: 3-SAT**
 - 3-SAT = $\{ \phi \mid \phi \text{ is a satisfiable function in 3-CNF} \}$
- Cook showed in 1971 that **SAT is an NP-complete** problem.
 - SAT is in NP, so any solution can be verified in polynomial time.
 - Every problem in NP can be polynomially reduced from SAT, which implies that if we could solve SAT in polynomial time, we could solve all NP problems in polynomial time.

Complexity of IPs

It is NP-hard to decide whether an IP has a feasible solution.

SAT is reducible to (binary) IPs.

Proof sketch:

- Let an instance of SAT be given by a conjunctive normal form of m clauses $C_1, \dots, C_m$ in boolean variables $x_1, \dots, x_n$.
- For each boolean variable x_i introduce a 0/1 variable v_i with $v_i = 1$ if $x_i = \text{TRUE}$.
- Then, the satisfiability of a clause can be written as a linear inequality and the existence of a satisfying assignment is equivalent to the existence of a feasible solution for the IP.
- For each clause $C = x_1 \vee \neg x_2 \vee \dots \vee x_i : v_1 + (1 - v_2) + \dots + v_i \geq 1$.

Theoretical computer science & complexity theory

1936 Turing develops a theoretical, mathematically based computability model

1960 Theory of formal languages and automata theory (Noam Chomsky)

1971 Steve Cook defines NP-complete problems

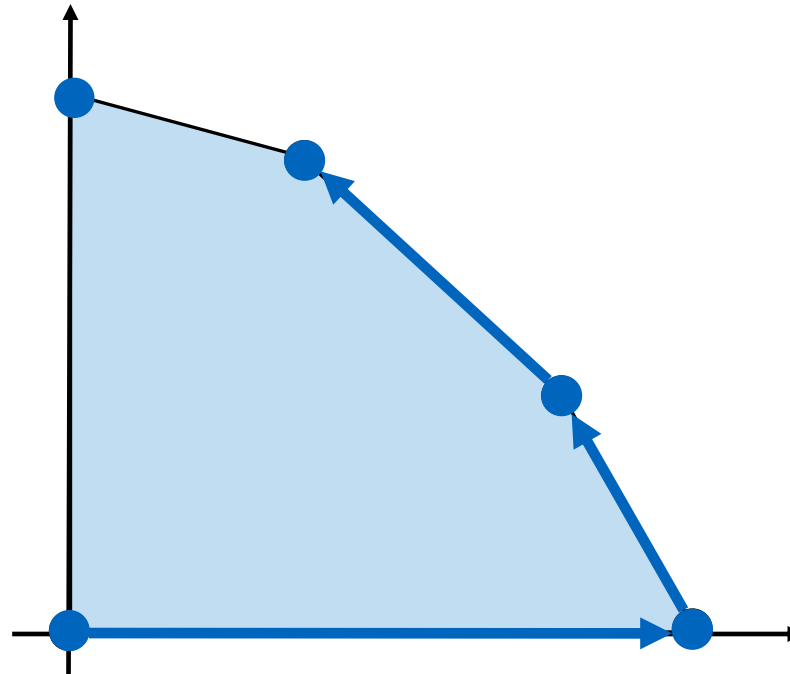
1972 Richard Karp shows that many combinatorial problems are NP-complete

since 1980: numerous approaches to develop fast algorithms for "hard" problems
and explore further/higher complexity classes, analysis of approximation
algorithms and online problems

LP: algorithms

Simplex. (Dantzig 1947)

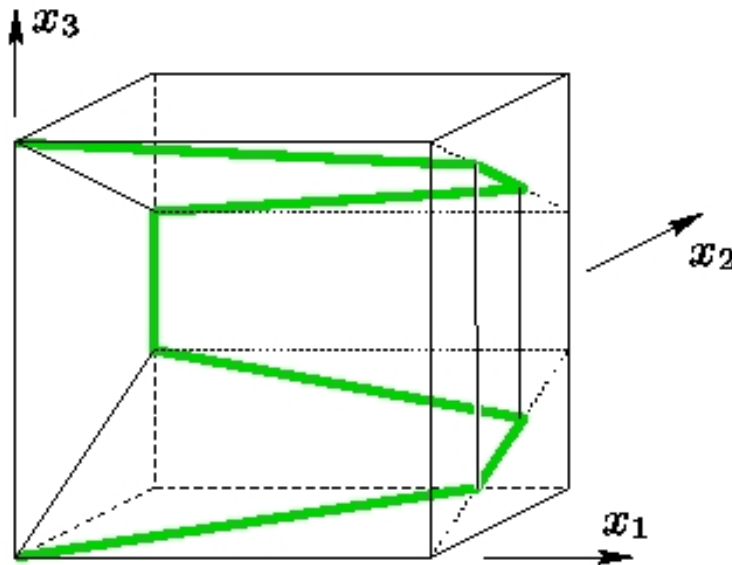
- developed after WW2 for logistical problems
- practical method, which searches adjacent extreme points
- exponential “worst-case” time complexity, polynomial implementations are not known (Klee & Minty, 1972)



Klee-Minty cube

The so-called Klee-Minty cubes are n -dimensional polytopes that look like slightly warped n -cubes and have the curious property that there is an ascending path through all 2^n vertices.

First "average-case" analysis by Borgwardt (1982) and later Haimovich (1983): $((n^4m)$ and $(n + m)$ pivot operations, respectively).



References:

- K.-H. Borgwardt. The average number of pivot steps required by the simplex-method is polynomial.
Zeitschrift für Operations Research. Serie A. Serie B, 26(5):A157.A177, 1982.
- V. Klee and G. J. Minty. How good is the Simplex algorithm? In *Inequalities, III* (Proc. Third Sympos., Univ. California, Los Angeles, Calif., 1969; dedicated to the memory of Theodore S. Motzkin), pages 159-175. Academic Press, New York, 1972.

LP: polynomial algorithms

Ellipsoid (Khachian 1979, 1980)

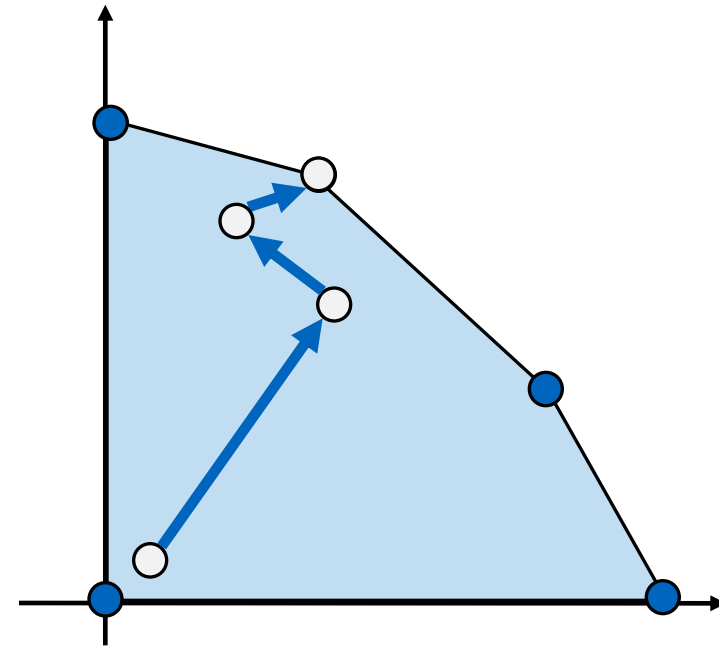
- solvable in polynomial time: $O(n^4 L)$,
n = # variables, L = coding length of the LP
- theoretically complex, practically of little importance and usually slower than simplex

Karmarkar's Algorithm (Karmarkar 1984)

- polynomial time ($O(n^{3.5} L)$)

... and later interior-point methods (barrier methods)

- many contributions since the 90s
- in most cases, comparable with simplex!
 - sometimes better for bigger problems and sparse matrices.
 - a disadvantage is that they are comparably unsuitable for solving a series of optimization problems (which is important for many algorithms of integer optimization, such as branch and bound or cutting plane methods).



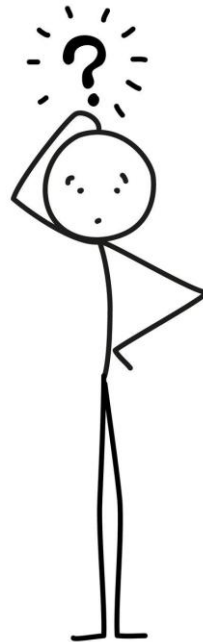
What method should be chosen?

Despite its exponential “worst-case” time, simplex is the most popular method for solving linear optimization problems.

Simplex provides a tool for sensitivity analysis and its runtime is competitive with newer approaches in practice.

For large LPs, it must be decided on a case-by-case basis which method to choose.

How can you model the binary knapsack problem as an integer optimization problem?



Known problems that can be represented as IP

- Assignment problem
- Knapsack problem
- Bin packing problem
- Set cover
- ...

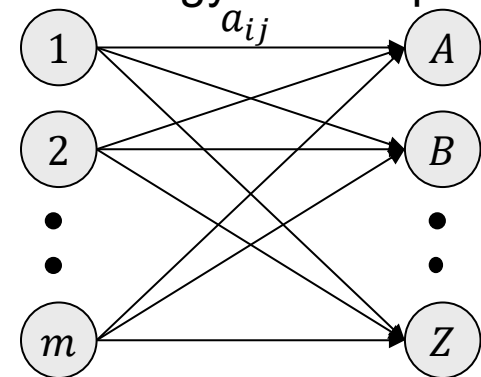
Example assignment problem/ bipartite matching problem

3 jobs can be performed on any of 3 machines.

The jobs must be assigned in such a way that the total energy consumption is minimal.

The individual machines have the following work-specific energy consumptions:

Job	Machine		
	<i>A</i>	<i>B</i>	<i>C</i>
A_1	8	5	9
A_2	14	10	12
A_3	15	13	16



How can energy consumption be minimized?

Solving by complete enumeration

Notation: $\{(Job, Machine)\} \Rightarrow \text{Energy}$

$$(1,1); (2,2) \text{ and } (3,3) \Rightarrow 8 + 10 + 16 = 34$$

$$(1,1); (2,3) \text{ and } (3,2) \Rightarrow 8 + 12 + 13 = 33$$

$$(1,2); (2,1) \text{ and } (3,3) \Rightarrow 5 + 14 + 16 = 35$$

$$(1,2); (2,3) \text{ and } (3,1) \Rightarrow 5 + 12 + 15 = 32 \text{ (Minimum!)}$$

$$(1,3); (2,1) \text{ and } (3,2) \Rightarrow 9 + 14 + 13 = 36$$

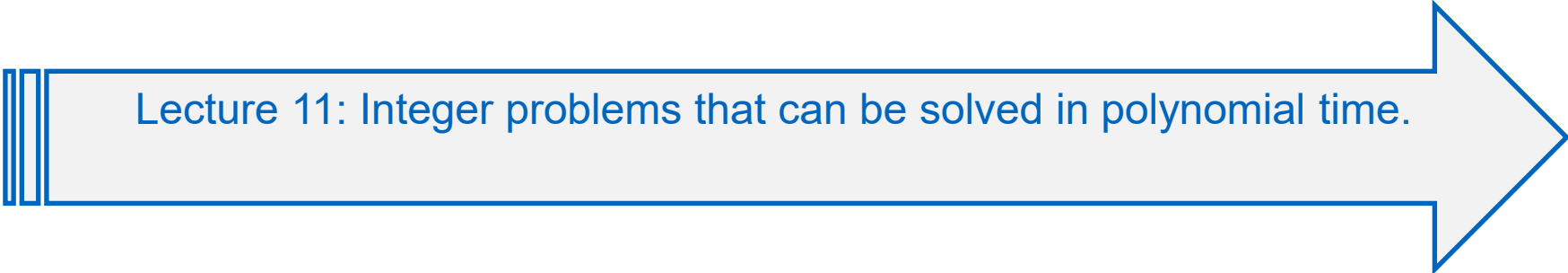
$$(1,3); (2,2) \text{ and } (3,1) \Rightarrow 9 + 10 + 15 = 34$$

→ Minimal energy consumption with the assignment:

Job	1	2	3
Machine	2	3	1

Features of the assignment problem

- Assignment problem := find a maximal matching in a weighted bipartite graph.
- Special case of the transportation problem with exactly n jobs and n machines.
- All constraints have right hand side equal to 1.
- The constraints are (in this case) equalities instead of inequalities.
- The value of all structure variables is 0 or 1.
- There are $n!$ possible solutions, with $n = \text{\#jobs} = \text{\#machines}$.
- Solution via the “Hungarian method” (in polynomial time).



Lecture 11: Integer problems that can be solved in polynomial time.

Generalized Assignment Problem (GAP)

Given:

- Jobs (B & W and color) $j \in J$, that can be processed independently on machines (e.g. printers) $i \in I$
- processing time d_{ij} of job j on machine i
- processing costs c_{ij}
- machine capacities s_i

Objective:

- Find a minimum cost and capacity feasible allocation of jobs to machines.

Modeling as a binary optimization problem

$$\begin{array}{ll} \min & \sum_{i \in I} \sum_{j \in J} c_{ij} x_{ij} \\ \text{s.t.} & \sum_{i \in I} x_{ij} = 1 \quad \text{for all jobs } j \in J \\ & \sum_{j \in J} d_{ij} x_{ij} \leq s_i \quad \text{for all machines } i \in I \\ & x_{ij} \in \{0, 1\} \quad \text{for all } i \in I, j \in J \end{array}$$

Known problems that can be represented as IP

- Assignment problem
- Knapsack problem
- Bin packing problem
- Set cover
- ...

0-1 (binary) Knapsack problem

- n projects, budget W
- w_j : cost of accepting a project j
- p_j : profit of project j
- x_j : 1, if project j is chosen, otherwise 0
- Objective: find the set of projects with the highest profit given the project budget.

$$\max \sum_{j \in J} p_j x_j$$

$$\text{s.t.} \quad \sum_{j \in J} w_j x_j \leq W$$

$$x_j \in \{0, 1\} \quad \text{for all } j \in J$$

Multiple Knapsack Problem

- n items
- m knapsacks with capacities $W_i, 1 \leq i \leq m$
- w_j : cost of item j
- p_j : value of item j
- x_{ij} : 1, if item j is taken in knapsack i , otherwise 0

$$\begin{aligned} \max \quad & \sum_{i \in I} \sum_{j \in J} p_j x_{ij} \\ \text{s.t.} \quad & \sum_{j \in J} w_j x_{ij} \leq W_i && \text{for all } i \text{ knapsacks} \\ & \sum_{i \in I} x_{ij} \leq 1 && \text{for all } j \text{ items} \\ & x_{ij} \in \{0, 1\} && \text{for all } i \text{ and } j \end{aligned}$$

Generalization of the knapsack problem

- Multiple knapsack problem
 - n items and m knapsacks
- Bounded knapsack problem
 - Each item is available in multiple copies.
- Multiple-choice knapsack problem
 - Items are partitioned into k classes and exactly one item from each class must be chosen.
- For further versions see:
 - http://en.wikipedia.org/wiki/List_of_knapsack_problems
 - Martello & Toth: Knapsack Problems. 1990.
 - Kellerer, Pferschy & Pisinger: Knapsack Problems. 2004.

Known problems that can be represented as IP

- Assignment problem
- Knapsack problem
- Bin packing problem
- Set cover
- ...

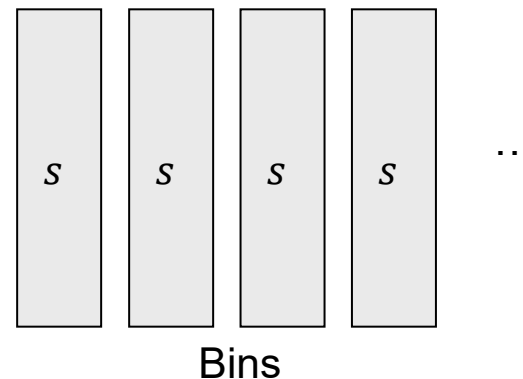
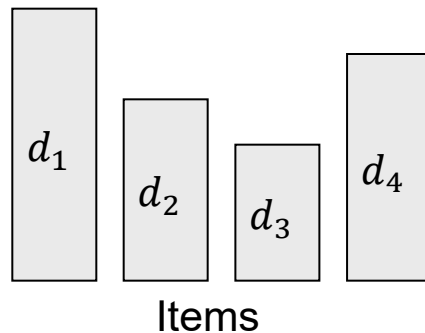
Bin Packing

Given:

- A set of items $j \in J$ with different volumes d_j and a set of bins with maximum capacity $s \geq \max_j d_j$.

Objective:

- Find an allocation of items to bins such that the capacity is not exceeded, and the number of bins is minimal.



Modeling as a binary optimization problem

Goal: minimize the number of bins i

$$\min \sum_{i \in I} y_i$$

$$\text{s.t. } \sum_{i \in I} x_{ij} = 1 \quad \text{for all } j \text{ items}$$

(Each item can be assigned exactly once.)

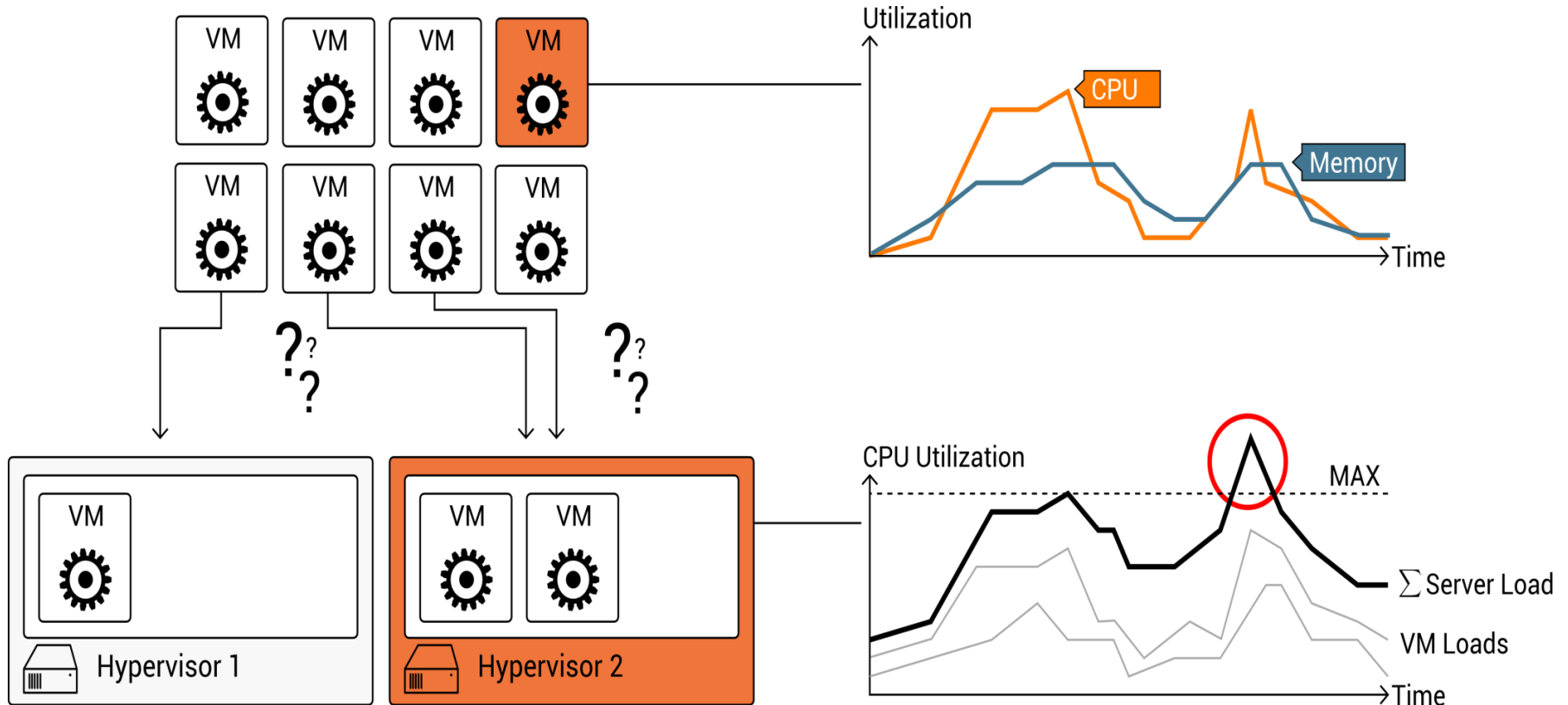
$$\sum_{j \in J} d_j x_{ij} \leq s y_i \quad \text{for all } i \text{ bins}$$

(The total capacity per bin is $\leq s$)

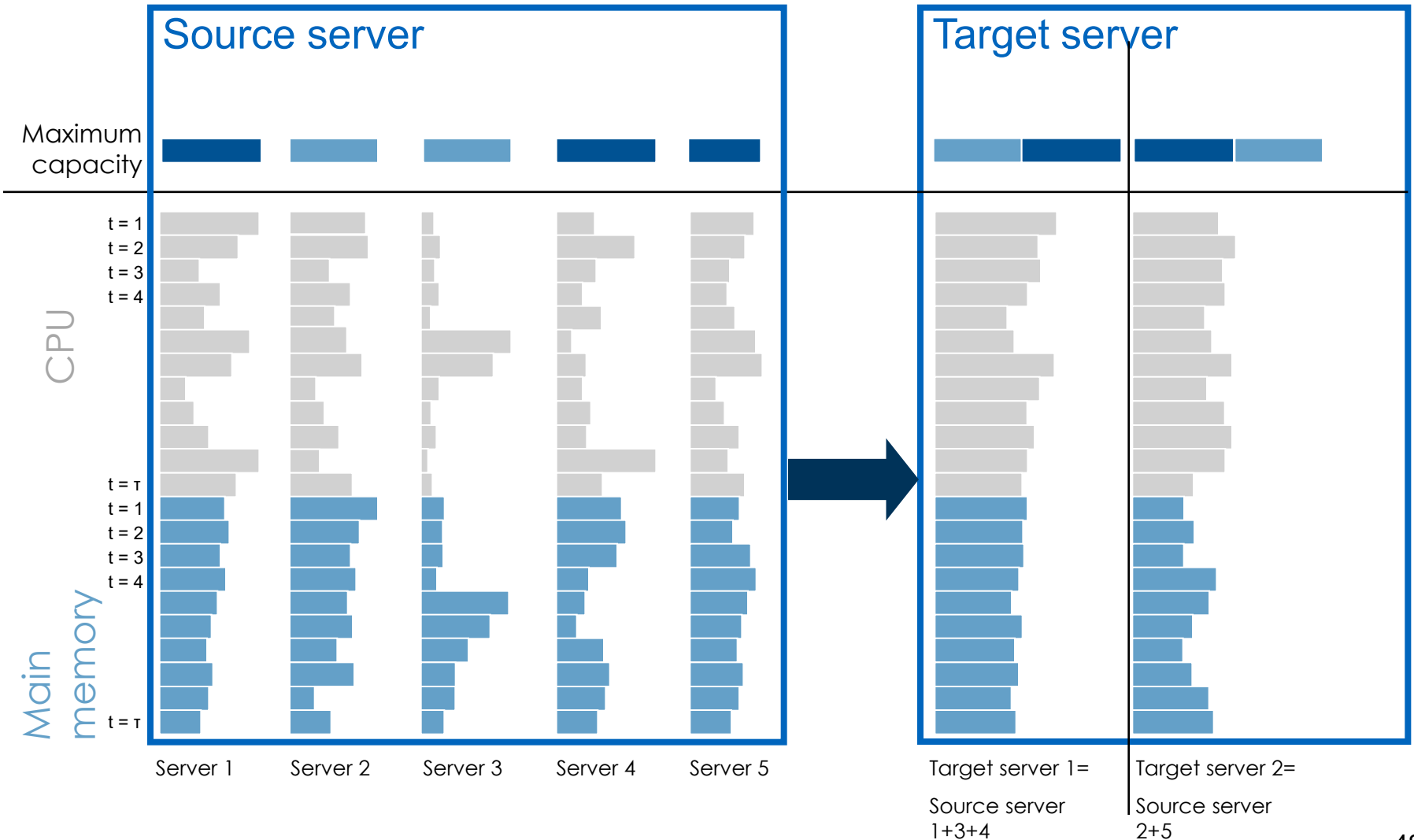
$$x_{ij}, y_i \in \{0, 1\} \quad \text{for all } i \text{ and } j$$

Server consolidation

Assignment of VMs to hypervisors



Multidimensional bin packing problem



Multidimensional bin packing problem

$$\begin{aligned}
 \min \quad & \sum_{i \in I} c_i y_i \\
 \text{s.t.} \quad & \sum_{i \in I} x_{ij} = 1 \quad \text{for all } j \in J \\
 & \sum_{j \in J} u_{jkt} x_{ij} \leq s_{ik} y_i \quad \text{for all } i \in I, k \in K, t \in T \\
 & x_{ij}, y_i \in \{0, 1\} \quad \text{for all } i \text{ and } j
 \end{aligned}$$

K – resources

T – time units

J – virtual machines (VMs)

I – physical server

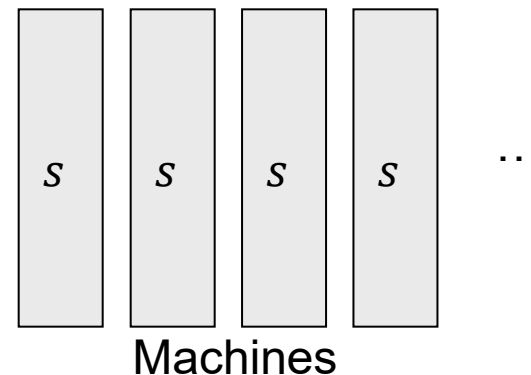
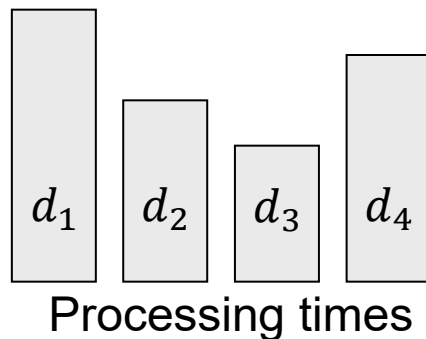
u – resource requirements

s – capacity limit of a server

Problem size for 5 VMs per server and 400 VMs:
 > 15.000 variables and > 60.000 constraints

Further applications

- Cutting-stock problems (Waste optimization)
- Planning truck loads
- Scheduling TV commercials with different lengths in pauses of 120 seconds
- Scheduling jobs on machines
 - weights d_i are processing times and
 - s are deadlines
- ...



Known problems that can be represented as IP

- Assignment problem
- Knapsack problem
- Bin packing problem
- Set cover
- ...

Set covering and partitioning

Set	Components	Costs
s_1	$\{a, c, d\}$	0.5
s_2	$\{b, c\}$	0.2
s_3	$\{b, e\}$	0.3
s_4	$\{a, d\}$	0.1
s_5	$\{c, e\}$	0.2
s_6	$\{b, c, e\}$	0.6
s_7	$\{c, d\}$	0.2

$$A = \begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 1 & 1 & 1 \\ 1 & 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 \end{bmatrix}$$

Optimal cover: $s_2, s_4, s_5 = 0.5$

Optimal partition: $s_4, s_6 = 0.7$

Set covering and partitioning

Given are m sets and n items:

$a_{ij} = 1$, if set j contains item i , otherwise 0

c_j = cost of set j

$x_j = 1$, if set j is included, otherwise 0

Set Covering:	Min	$c^T x$
	s.t.	$Ax \geq 1, x \text{ binary}$
Set Partitioning:	Min	$c^T x$
	s.t.	$Ax = 1, x \text{ binary}$
Set Packing:	Max	$c^T x$
	s.t.	$Ax \leq 1, x \text{ binary}$

Combinatorial procurement auction

Procurement: Sugar	Suppliers			
	L_1	L_2	L_3	L_4
100 t in Augsburg	30	80	100	30
5 t in Munich	0	5	5	0
20 t in Vienna	20	10	20	10
Bid (in thousands)	€ 150	€ 125	€ 300	€ 125

Buyer specifies the demand for several goods.

Suppliers bid on bundles of goods (here with several goods).

Selection of bids that meet the need at the lowest cost.

Allocation of bids

Procurement: Sugar	Suppliers			
	L_1	L_2	L_3	L_4
100 t in Augsburg	30	80	100	30
5 t in Munich	0	5	5	0
20 t in Vienna	20	10	20	10
Bid (in thousands)	€ 150	€ 125	€ 300	€ 125

Binary variable	x_1	x_2	x_3	x_4
-----------------	-------	-------	-------	-------

$$\begin{aligned}
 &\text{Minimize } 150 x_1 + 125 x_2 + 300 x_3 + 125 x_4 \\
 &\text{s.t. } 30 x_1 + 80 x_2 + 100 x_3 + 30 x_4 \geq 100 \\
 &\quad \quad \quad 5 x_2 + 5 x_3 \geq 5 \\
 &\quad \quad \quad 20 x_1 + 10 x_2 + 20 x_3 + 10 x_4 \geq 20
 \end{aligned}$$

Solution: $x_2 = 1$ and $x_4 = 1 \rightarrow \text{Price} = 125 \text{ €} + 125 \text{ €} = 250 \text{ €}$

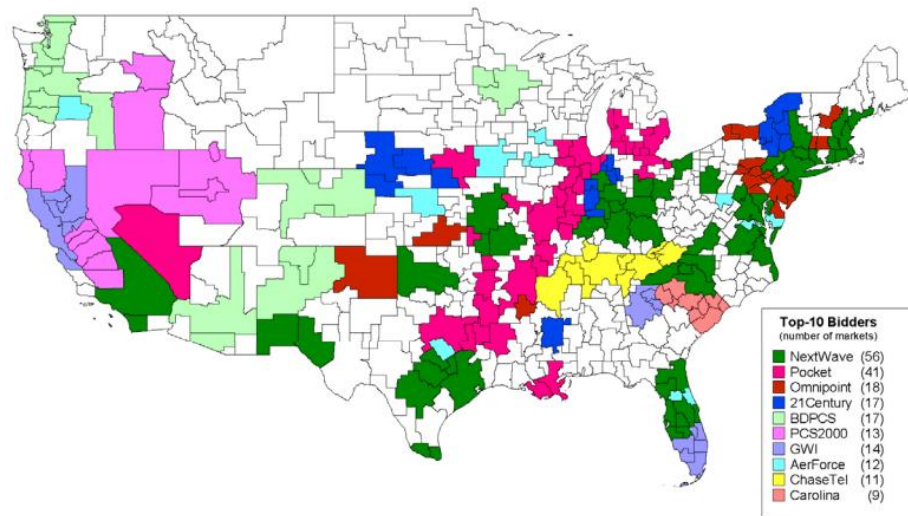
$$\begin{aligned}
 &\min \sum_j p_j x_j \\
 &\text{s.t. } \sum_j w_{ij} x_j \geq W_i, \text{ for all } i \\
 &\quad \quad \quad x_j \in \{0, 1\}
 \end{aligned}$$

Combinatorial auctions for allocating frequency licenses

Since the mid-1990s, frequency licenses have been auctioned off, usually in a simultaneous ascending auction.

Since 2008, combinatorial auctions have been used worldwide for the allocation of frequency licenses.

In such a sales auction, the set covering problem is replaced by a set packing problem.



Further applications of set covering

- Crew scheduling:
 - Each set is a route for a specific crew (e.g., MUC→VIE→HAM).
 - Each item is a flight to be crewed.
- Planning public facilities:
 - Each set is a facility (e.g., warehouse, fire department, nursery) that serves specific areas.
 - Each item is an area to be supplied.

Planning public facilities (Facility Location)

- There are 6 cities in a region.
- The administration must establish disaster response centers for the region.
- A minimum number of centers should be established where at least 1 center can be reached from each city within 15 minutes.

Travel times between cities

	S_1	S_2	S_3	S_4	S_5	S_6
S_1	0	10	20	30	30	20
S_2	10	0	25	35	20	10
S_3	20	25	0	15	30	20
S_4	30	35	15	0	15	25
S_5	30	20	30	15	0	14
S_6	20	10	20	25	14	0

Summary

Integer programs are suitable for modeling many real applications.

Especially for binary decisions and logical operations a formulation as a real-valued LP is often not sufficient.

However, the price is the high complexity (NP-hard).

Procedures for LPs are not appropriate to obtain optimal solutions.

→ We need new methods [*next lecture*]:

- exact methods
- approximation methods (approximation / heuristics)